

Discrete Ordinates Method

Parts One and Two: Full DOM Solver for a Leaf Canopy

A Companion Guide to dom_student.py

Physical Models in Remote Sensing — Chapter 04, Parts 03 and 04

How To Use This Document

1. Read Sections 1 and 2 of this document (problem and geometry).
2. Open dom_student.py and go to Section 0 to set your inputs.
3. Run: `python dom_student.py`
Answer 0 (no interpolation) to get the energy balance table only.
Sanity check: for RED LAI=1.5 you should see "Converged in 3 iterations"
and "imbalance = +0.095%". A different iteration count or large imbalance
indicates a problem — fix it before writing your own code.
4. Come back and read each section as you work through the code.

1. The Problem We Are Solving

1.1 What Is a Leaf Canopy?

A leaf canopy is a layer of vegetation through which sunlight passes before reaching the ground. Sunlight enters from above, interacts with leaves (reflecting and transmitting through them many times), and some fraction eventually escapes back upward. Remote sensing instruments measure this upward-escaping light to infer vegetation properties such as leaf area index and chlorophyll content.

1.2 The Two-Part Strategy (C4-P3, Slide 3)

Modelling radiation is complicated because photons scatter off leaves repeatedly. We split the total field I into two independent parts:

Total radiation field: $I(L, \Omega) = I_0(L, \Omega) + I_C(L, \Omega)$
I_0 = UNCOLLIDED radiation
Photons that have NOT yet hit any leaf.
Travel in straight lines -> Beer-Lambert attenuation only.
Solved in Steps 1-6 (C4-P3) -> code Sections 1-5.
I_C = COLLIDED radiation
Photons that have hit at least one leaf and scattered.
Require an iterative solver.
Solved in Steps 7-11 (C4-P4) -> code Section 6.

1.3 The Governing Equations (C4-P3, Slides 4-5)

Uncollided field (no scattering source):

$-\mu * dI_0/dL + G * I_0(L, \Omega) = 0$
Solution: Beer-Lambert $I_0(L_2) = I_0(L_1) * \exp[-G*(L_2-L_1)/ \mu]$

Collided field (driven by scattering sources Q and S):

$-\mu * dI_C/dL + G * I_C(L, \Omega) = Q(L, \Omega) + S(L, \Omega)$
Q = First Collision Source (first scatter of uncollided photons)

S = Multiple Scattering Source (re-scatter of already-collided photons)
Boundary conditions:
IC(L=0, Omega) = 0 for mu < 0 (nothing enters from above)
IC(L=LAI, Omega) = Lambertian reflection of downward collided flux for mu > 0

2. Coordinate System and Sign Convention (C4-P3, Slide 2)

2.1 The Spatial Variable L

Zenith (-z, pointing UP)
L = 0 +----- canopy TOP (radiation enters here)
leaf canopy
L = LAI +----- GROUND (boundary condition here)
///

L increases downward. $k=1$ is the top ($L=0$), $k=K+1$ is the ground ($L=LAI$). Cell centres are at $k+\frac{1}{2}$ for $k=1..K$. Arrays have size $K+2$; index 0 is a dummy never used.

2.2 The Direction Variable Omega = (theta, phi)

$\mu = \cos(\theta)$, θ measured from the UPWARD zenith.
$\mu < 0 \Rightarrow$ DOWNWARD direction (sun, sky, downward diffuse)
$\mu > 0 \Rightarrow$ UPWARD direction (reflected, scattered upward)
Example: $\theta_{\text{solar}} = 140 \text{ deg} \Rightarrow \mu_{\text{solar}} = \cos(140) = -0.766$
Quadrature index: $n = 1..N/2$ downward ($\mu < 0$)
$n = N/2+1..N$ upward ($\mu > 0$)
ALL arrays are 1-based. Index 0 exists but is never used.

3. Algorithm Overview

The solver runs 11 numbered steps across both parts of the lecture slides. The complete flow:

INPUT: LAI, f_dir, theta_solar, phi_solar, rho_g, rho_L, tau_L
N (polar), M (azimuth), K (layers), TOL, MAX_ITER
v
STEP 1: Angular discretization [Section 1]
N Gauss-Legendre nodes mu[1..N], weights w_mu[1..N]
M uniform azimuth angles phi[1..M], weight w_phi = 2*pi/M
Pre-compute G_qq[n,m,i,j] and G_sol[i,j] (phase function tables)
v
STEP 2: Spatial discretization [inside solve_one_case]
K layers, dL = LAI/K (just two lines: G = G_extinction(); dL = LAI/K)
v
STEP 3a: Downward sweep - direct solar [Section 4, solve_uncollided]
STEP 3b: Downward sweep - diffuse sky [Section 4, solve_uncollided]
v
STEP 4: Lower BC uncollided [Section 4, solve_uncollided]
v
STEP 5: Upward sweep [Section 4, solve_uncollided]
v
STEP 6: First Collision Source Q = Q1+Q2+Q3 [Section 5]
v
STEP 7-9: DD sweep equations, coefficients [Section 6, solve_collided]

v
STEP 10: ITERATION LOOP (dual convergence) [Section 6, solve_collided]
1) S=0 first. J=Q+S. Sweep down. Lower BC from THIS sweep. Sweep up.
2) Compute new S. Check dual convergence (A: boundary IC, B: scalar irradi).
3) If not converged, goto 1.
v
STEP 11: Energy Balance [Section 7]
Compute scalar irradiance S(k). Abs = G*(1-omL)*dL*sum(S).
Ref + Trans + Abs_leaves = 1 (Trans=Abs_soil; genuine diagnostic)
v
OUTPUT: Word document with energy balance table.
Optional: BRF/HDRF table at user-defined angular grid.

4. Step 1: Angular Discretization (C4-P3, Slides 7-9) — Code Section 1

4.1 Gauss-Legendre Polar Quadrature

<code>nodes, weights = np.polynomial.legendre.leggauss(N)</code>
<code># 1-based storage (index 0 = dummy, never used):</code>
<code>mu[1:N+1] = nodes # ascending order: negative first, then positive</code>
<code>w_mu[1:N+1] = weights</code>
<code>Downward: n = 1 .. N/2 (mu[n] < 0)</code>
<code>Upward: n = N/2+1 .. N (mu[n] > 0)</code>
<code>Weight checks (Slide 9):</code>
<code>sum(w_mu[1..N]) = 2.0 (integral of 1 over [-1,+1])</code>
<code>sum(mu[N/2+1..N]*w_mu[..]) ~ 0.5 (upward hemisphere flux)</code>

4.2 Azimuthal Directions

<code>phi[m] = (m-1) * 2*pi/M for m = 1..M</code>
<code>w_phi = 2*pi / M (scalar, same for every azimuth)</code>

4.3 Extinction Coefficient G (C4-P3, Slide 7) — Code Section 2

The slides assume a *uniform leaf normal orientation* (all leaf-normal directions equally probable), which gives $G = 0.5$ for every propagation direction. Physically, $G = 0.5$ is the mean projected area of a randomly-oriented flat plate viewed from any direction — it equals the projected area of a sphere of unit cross-section, and is direction-independent for the uniform distribution. The function `G_extinction()` simply returns 0.5. If you later extend the code to a non-uniform leaf distribution, only this function needs to change.

4.4 Phase Function Tables

Two tables are pre-computed once per spectral band (not per LAI, not per iteration):

<code>G_qq[n, m, i, j] = Gamma((mu_n, phi_m) -> (mu_i, phi_j))</code>	<code>shape (N+1, M+1, N+1, M+1)</code>
<code>G_sol[i, j] = Gamma(solar -> (mu_i, phi_j))</code>	<code>shape (N+1, M+1)</code>

```
For N=16, M=16: 65,536 Gamma evaluations.
```

```
Computed ONCE, reused for all LAI values, all iterations.
```

4.5 Step 2: Spatial Discretization (C4-P3, Slide 10) — inside `solve_one_case`

Step 2 has no dedicated function. It is just two lines near the top of `solve_one_case`:

```
G = G_extinction()    # = 0.5 for uniform leaf-angle distribution
```

```
dL = LAI / K          # layer thickness
```

The canopy is divided into K uniform layers of thickness $\Delta L = LAI/K$ (Slide 10). Cell *edges* are indexed $k = 1$ (canopy top, $L = 0$) to $k = K+1$ (ground, $L = LAI$). Cell *centres* are at $k+\frac{1}{2}$ for $k = 1 \dots K$ and are used for the scattering sources Q and S . All intensity arrays are allocated with $K+2$ entries; index 0 is a dummy that is never used. The spatial grid is fully described by the two scalars G and dL — there is no grid array.

5. Steps 3a, 3b: Downward Sweeps (C4-P3, Slides 12-13) — Code Section 4

5.1 Upper Boundary Condition (C4-P3, Slide 4)

Direct solar: $I_{\text{beam}} = f_{\text{dir}} * F_{\text{in}} / \mu_{\text{solar}} $
(irradiance $f_{\text{dir}} * F_{\text{in}}$ divided by $ \mu $ gives intensity)
Diffuse sky: $I_{\text{sky}} = (1 - f_{\text{dir}}) * F_{\text{in}} / \pi$
(isotropic radiance at the canopy top)

5.2 Step 3a: Direct Solar Recurrence Sweep (Slide 12)

exp_step_sol computed ONCE outside the loop:
$\text{exp_step_sol} = \exp(-G * dL / \mu_{\text{solar}})$
$I0_{\text{dir}}[1] = I_{\text{beam}}$ # upper BC: k=1
for k in range(1, K+1):
$I0_{\text{dir}}[k+1] = I0_{\text{dir}}[k] * \text{exp_step_sol}$ # recurrence downward
Done only for the ONE solar direction.

5.3 Step 3b: Diffuse Sky Recurrence Sweep (Slide 13)

for n in range(1, N/2+1): # downward directions: $\mu[n] < 0$
$\text{exp_step_n} = \exp(-G * dL / \mu[n])$
for m in range(1, M+1):
$I0[n,m,1] = I_{\text{sky}}$ # upper BC: k=1
for k in range(1, K+1):
$I0[n,m,k+1] = I0[n,m,k] * \text{exp_step_n}$
Done for ALL $(N/2)*M$ downward directions.

6. Step 4: Lower Boundary Condition Uncollided (C4-P3, Slide 14) — Code Section 4

```
# Total downward flux at the ground (BOTH diffuse AND direct):
```

```
F_down_gnd = Fd_dif[K+1] + Fd_dir[K+1]
```

```
# Lambertian reflection: same intensity in all upward directions:
```

```
I0_up_gnd = (rho_g / pi) * F_down_gnd
```

```
# Set as lower BC for the upward sweep (Step 5):
```

```
I0[n, m, K+1] = I0_up_gnd    for n = N/2+1..N, all m
```

CRITICAL: F_down_gnd must include BOTH Fd_dif[K+1] AND Fd_dir[K+1].

Slide 14 shows two terms: the sum over downward Gauss directions

(diffuse) plus $|\mu_o| \cdot I0_dir[K+1]$ (direct beam).

A common error is to include only the diffuse term.

Note: $Fd_dir[K+1] = |\mu_solar| \cdot I0_dir[K+1]$. That is, the direct-beam

downward irradiance = beam intensity * $|\mu_solar|$. This is exactly the

second term $|\mu_o| \cdot I0dir(Lk=K+1, \Omega_o)$ shown on Slide 14.

7. Step 5: Upward Sweep (C4-P3, Slide 16) — Code Section 4

```
for n in range(N/2+1, N+1):      # upward directions: mu[n] > 0

    exp_step_n = exp(-G * dL / |mu[n]|)

    for m in range(1, M+1):

        I0[n,m,K+1] = I0_up_gnd  # lower BC from Step 4

        for k in range(K, 0, -1):

            I0[n,m,k] = I0[n,m,k+1] * exp_step_n  # recurrence upward

        
```

Done for ALL (N/2)*M upward directions.

8. Step 6: First Collision Source Q (C4-P3, Slides 18-20) — Code Section 5

8.1 Cell-Centre Rule

Cell-centre intensity = MEAN of surrounding edges (used everywhere):		
$I0_dir_c[k]$	$= 0.5 * (I0_dir[k] + I0_dir[k+1])$	for $k=1..K$
$I0_c[n,m,k]$	$= 0.5 * (I0[n,m,k] + I0[n,m,k+1])$	for $k=1..K$
$IC_c[n,m,k]$	$= 0.5 * (IC[n,m,k] + IC[n,m,k+1])$	for $k=1..K$

8.2 Q1: Direct Solar (Slide 18)

$Q1[i,j,k] = (1/\pi) * G_sol[i,j] * I0_dir_c[k]$
Output: ALL N*M directions $i=1..N$, $j=1..M$, all K cell centres.

8.3 Q2: Downward Diffuse Sky (Slide 19)

$Q2[i,j,k] = (1/\pi) * \sum_{n=1}^{N/2} \sum_{m=1}^M$
$w_mu[n] * w_phi * G_qq[n,m,i,j] * I0_c[n,m,k]$
Source sum: $n = 1..N/2$ (DOWNWARD only).
Output: $i = 1..N$ (all directions, both hemispheres).

8.4 Q3: Ground-Reflected Uncollided (Slide 20)

$Q3[i,j,k] = (1/\pi) * \sum_{n=N/2+1}^N \sum_{m=1}^M$
$w_mu[n] * w_phi * G_qq[n,m,i,j] * I0_c[n,m,k]$
Source sum: $n = N/2+1..N$ (UPWARD only).
Output: $i = 1..N$ (all directions).
$Q[i,j,k] = Q1 + Q2 + Q3$ (total First Collision Source)

9. Steps 7-9: The Collided RTE and DD Coefficients (C4-P4, Slides 4-6) — Code Section 6

9.1 Multiple Scattering Source S (Slide 4)

<code>S[i,j,k] = (1/pi) * sum_{n=1}^{N} sum_{m=1}^{M}</code>
<code>w_mu[n] * w_phi * G_qq[n,m,i,j] * IC_c[n,m,k]</code>
Sum over ALL N directions (both hemispheres) -- unlike Q2 and Q3.
S=0 on the first iteration (IC=0).

9.2 Diamond-Difference Coefficients (Slide 6) — SIGNED f

<code># f is defined with the SIGNED mu[n], not mu[n] :</code>
<code>f[n] = G * dL / mu[n] # negative for downward, positive for upward</code>
<code># With alpha = 0.5:</code>
<code>a[n] = (1 + (1-alpha)*f[n]) / (1 - alpha*f[n])</code>
<code>b[n] = f[n] / (G * (1 - alpha*f[n]))</code>
<code>c[n] = (1 - (1-alpha)*f[n]) / (1 + alpha*f[n])</code>
<code>d[n] = f[n] / (G * (1 + alpha*f[n]))</code>
For downward (mu < 0): f < 0 => a < 1, b < 0
For upward (mu > 0): f > 0 => c < 1, d > 0

CRITICAL: f MUST be computed with signed mu[n], not mu[n] .
Using mu[n] makes f > 0 for downward directions, giving a > 1.
The sweep equation IC(k+1) = a*IC(k) - b*J then SUBTRACTS the source
J instead of adding it -- physically wrong.
With signed f (f < 0 for downward): b < 0, so -b*J = + b *J adds J. Correct.

The two codes (dom_student.py and dom_rt.py) both use signed f .

NOTE ON CODE DOCSTRING: The internal docstring of solve_collided() uses a simplified notation: $\text{half_f} = 0.5 * G * dL / |\mu|$, writing the downward sweep as

$$IC(k+1) = c * IC(k) + dd * J \quad \text{where } c = (1 - \text{half_f}) / (1 + \text{half_f}).$$

This is IDENTICAL to $a[n] * IC(k) - b[n] * J$ used in the actual code when $\alpha = 0.5$. Substituting $\alpha = 0.5$ into the general a, b formulas gives

$c_docstring = a_code$ and $dd_docstring = -b_code$.

They are two ways of writing exactly the same formula.

9.3 Sweep Equations (Slide 6)

Downward ($\mu[n] < 0$), sweep $k = 1 \rightarrow K$:

$$IC(k+1) = a[n] * IC(k) - b[n] * J(k) \quad \# \text{ upper BC: } IC[n, m, 1] = 0$$

Apply lower BC from the CURRENT sweep (see Section 10):

$$F_{\text{col_gnd}} = \sum_{n=1..N/2} w_{\mu}[n] * w_{\phi} * |\mu[n]| * IC_{\text{new}}[n, m, K+1]$$
$$IC_{\text{gnd_refl}} = (\rho_g / \pi) * F_{\text{col_gnd}}$$

Upward ($\mu[n] > 0$), sweep $k = K+1 \rightarrow 1$:

$$IC(k) = c[n] * IC(k+1) + d[n] * J(k) \quad \# \text{ lower BC: } IC[n, m, K+1] = IC_{\text{gnd_refl}}$$

10. Step 10: The Iteration Loop and Convergence (C4-P4, Slide 7) — Code Section 6

10.1 Why Iteration Is Needed

The multiple scattering source S depends on IC , but IC is what the sweep computes from S . This circular dependency is resolved by source iteration: start with $S=0$, sweep to get IC , compute new S from IC , repeat until both are stable.

10.2 Correct Iteration Order

Before loop: $IC = 0, S = 0, IC_{prev} = 0, SI_{prev} = 0$
EACH ITERATION:
1. $J = Q + S$
2. Sweep DOWN: $IC_{new}[n,m,1]=0; IC_{new}[n,m,k+1] = a*IC_{new}[n,m,k] - b*J$
3. Lower BC: F_{col_gnd} from $IC_{new}[n,m,K+1]$ (THIS sweep, not previous)
$IC_{gnd_refl} = (\rho_g/\pi)*F_{col_gnd}$
4. Sweep UP: $IC_{new}[n,m,K+1]=IC_{gnd_refl}; IC_{new}[n,m,k] = c*IC_{new}[n,m,k+1] + d*J$
5. $IC = IC_{new}$
6. Compute S_{new} from IC (cell-centre means, all $N*M$ directions)
7. Check DUAL convergence (see Section 10.3)
8. If converged: stop. Else: $S = S_{new}, IC_{prev} = IC$, go to 1.

CRITICAL: Step 3 must use IC_{new} from the CURRENT downward sweep,
not IC from the previous iteration.
Using the previous IC for the ground BC causes incorrect convergence
behaviour and an inaccurate interior radiation field.

10.3 Dual Convergence Criterion (matching dom_rt.py)

Both criteria must be satisfied simultaneously before the loop stops.

CRITERION A: max relative change in IC at the BOUNDARY edges
For $k = 1$ and $k = K+1$, for directions with $ \mu[n] \geq \cos(70 \text{ deg})$:
$\text{crit_A} = \max IC[n,m,k] - IC_{prev}[n,m,k] / IC[n,m,k] $

Must be < TOL.
CRITERION B: max relative change in collided SCALAR IRRADIANCE profile
$SI_new[k] = \sum_{\{n,m\}} w_mu[n] * w_phi * IC_c[n,m,k]$ (no $ \mu $ weight)
$crit_B = \max_k SI_new[k] - SI_prev[k] / SI_new[k] $
Must be < TOL.
BOTH $crit_A < TOL$ AND $crit_B < TOL$ must hold simultaneously.

Why dual convergence?
Criterion A ensures the BOUNDARY fluxes (Ref, Trans) have converged.
Criterion B ensures the INTERIOR radiation density field has converged.
The scalar irradiance energy balance ($Abs = G * (1 - \omega_L) * dL * \sum(S)$) uses
the interior field directly. Without criterion B, NIR cases stop too early
(4-5 iterations instead of 8-15) and give imbalances of -40% to -60%.
With dual convergence, all four standard cases match dom_rt.py exactly:
RED LAI=1.5: +0.095% RED LAI=4.0: +0.131%
NIR LAI=1.5: -0.580% NIR LAI=4.0: -1.972%

10.4 Why the Iteration Converges

Source iteration converges because each scatter removes a fraction $(1 - \omega_L)$ of photon energy through leaf absorption. After n scatters, the remaining energy is proportional to ω_L^n . Since $\omega_L < 1$ for any real material, this geometric series converges. Convergence is slower for NIR ($\omega_L \sim 0.975$) because only 2.5% of energy is removed per scatter.

11. Step 11: Energy Balance (C4-P4, Slides 8-9) — Code Section 7

11.1 Scalar Irradiance (the same method as dom_rt.py)

The total leaf absorption is computed via the scalar irradiance integral, matching the approach used in the SOS full radiative transfer code and in dom_rt.py:

Scalar irradiance at cell centre $k+1/2$:
$S[k] = \sum_{n=1}^N \sum_{m=1}^M$
$w_{\mu}[n] * w_{\phi} * (I0_c[n,m,k] + IC_c[n,m,k]) \quad (\text{no } \mu \text{ weight})$
$+ I0_dir_c[k] \quad (\text{direct beam})$
Leaf absorption:
$Abs_leaves = G * (1 - \omega_L) * dL * \sum_{k=1}^K S[k] / F_{in}$
This is independent of Ref and Trans.

11.2 Full Energy Balance

Ref	= Fu_total[k=1] / F_in
	upward flux leaving the canopy top
Trans	= (Fd_total[K+1] - Fu_total[K+1]) / F_in
	net downward flux at the ground surface (= Abs_soil)
Abs_leaves	= G*(1-omega_L)*dL*sum(S) / F_in
	leaf absorption via scalar irradiance integral
Abs_soil	= (1-rho_g)*Fd_total[K+1] / F_in
	soil absorption (shown for completeness; = Trans)
Energy balance:	Ref + Trans + Abs_leaves = 1.0

NOTE: $\text{Trans} = \text{Abs_soil}$ identically (Lambertian ground: $\text{Fu}[K+1] = \rho_g * \text{Fd}[K+1]$).
Why: $\text{Trans} = \text{Fd}[K+1] - \text{Fu}[K+1] = \text{Fd}[K+1] - \rho_g * \text{Fd}[K+1] = (1 - \rho_g) * \text{Fd}[K+1]$
$= \text{fraction absorbed by soil} * \text{flux at soil} = \text{Abs_soil}$.
"Transmittance" here means the net flux entering the soil surface (not the
fraction passing through leaves). It equals soil absorption because the
Lambertian ground reflects upward; what is not reflected is absorbed.
Do NOT add both Trans and Abs_soil -- that double-counts the soil (gives ~1.33).
Imbalance (%) = $(\text{Ref} + \text{Trans} + \text{Abs_leaves} - 1) * 100$
Genuine diagnostic -- not a tautology.

Why scalar irradiance, not flux divergence?
Both methods are physically correct. The scalar irradiance method is
preferred because it matches dom_rt.py and the SOS code exactly, and
computes leaf absorption from the fundamental photon interaction rate:
$\text{absorption rate per unit LAI} = G * (1 - \omega_L) * S(L)$
The flux divergence method ($\text{Abs} = \text{net_flux_top} - \text{net_flux_bottom} - \text{Abs_soil}$)
is also correct, but the scalar irradiance method requires the DUAL
convergence criterion (Section 10.3) to be numerically accurate.
Without dual convergence the scalar irradiance gives -40% to -60% for NIR.
Expected imbalances at N=16, K=50, TOL=0.01:
RED LAI=1.5: +0.095% RED LAI=4.0: +0.131%
NIR LAI=1.5: -0.580% NIR LAI=4.0: -1.972%
(GL quadrature limitation at N=16; not a code error)

12. Interpolation into an Arbitrary View Direction (C4-P4, Slides 18-23) — Code Section 8

Function `dom_interpolate_view(theta_v_deg, phi_v_deg, ...)` computes the total intensity at canopy top in any upward view direction Ω_V using the converged Gauss-direction solution. Nine steps:

Step 1: Cross-sections $G(\Omega_V)$, $\Gamma(\text{solar} \rightarrow \Omega_V)$, $\Gamma(\text{nm} \rightarrow \Omega_V)$
Step 2: Uncollided ground BC in Ω_V
$I0_gnd_V = (\rho_g/\pi) * [\sum_n w_n * \mu_n * I0[n,m,K+1] + \mu_0 * I0_dir[K+1]]$
Step 3: Uncollided intensity at top (single Beer-Lambert sweep)
$I0_top_V = I0_gnd_V * \exp(-G * LAI / \mu_V)$
Step 4: $Q1, Q2, Q3$ in Ω_V (output direction is μ_V, ϕ_V throughout)
$Q1_V[k] = (1/\pi) * \Gamma_{sol_V} * I0_dir_c[k]$
$Q2_V[k] = (1/\pi) * \sum_{n=1..N/2} w_n * \mu_n * \Gamma_{nm_V}[n,m] * I0_c[n,m,k]$
$Q3_V[k] = (1/\pi) * \sum_{n=N/2+1..N} w_n * \mu_n * \Gamma_{nm_V}[n,m] * I0_c[n,m,k]$
Step 5: S in Ω_V using CONVERGED IC (no re-iteration needed)
$S_V[k] = (1/\pi) * \sum_{n=1..N} w_n * \mu_n * \Gamma_{nm_V}[n,m] * IC_c[n,m,k]$
Step 6: $J_V[k] = Q_V[k] + S_V[k]$
Step 7: Collided ground BC in Ω_V
$IC_gnd_V = (\rho_g/\pi) * \sum_{n=1..N/2} w_n * \mu_n * IC[n,m,K+1]$
Step 8: Single upward DD sweep in Ω_V
$IC_edge[K+1] = IC_gnd_V$
for $k = K..1$: $IC_edge[k] = c_V * IC_edge[k+1] + d_V * J_V[k]$

Step 9: $I_{\text{total}} = I0_{\text{top_V}} + IC_{\text{edge}}[1]$

BRF or HDRF = $I_{\text{total}} / (F_{\text{in}}/\pi)$

13. Expected Results

Standard inputs: N=16, M=16, K=50, TOL=0.01, LAI=1.5, RED band, theta_solar=140 deg, f_dir=0.7.

Console output should show:

iter 1 (initial sweep, S=0)
iter 2 bnd=6.23e-02 S=5.88e-02
iter 3 bnd=3.66e-03 S=3.49e-03 ✓
Converged in 3 iterations.
imbalance = +0.095%

Energy balance table should show:

Quantity	Uncollided	Collided	Total
Reflectance	0.01102	0.02099	0.03201
Transmittance (net)	--	--	0.33202
Leaf Absorption	--	--	0.63693
Soil Absorption	--	--	0.33213
SUM Ref+Trans+Leaf	--	--	1.00095
Energy Imbal. (%)			+0.095

For all four standard cases the imbalances are:

RED LAI=1.5: +0.095%	RED LAI=4.0: +0.131%
NIR LAI=1.5: -0.580%	NIR LAI=4.0: -1.972%
The NIR imbalance is a GL quadrature limitation at N=16, not a code error.	
It exactly matches dom_rt.py and is consistent with the SOS code behaviour.	

14. Common Errors and Their Symptoms

Error 1 — Wrong upper BC for the direct beam

Wrong: $I_{\text{beam}} = f_{\text{dir}} * F_{\text{in}}$
Correct: $I_{\text{beam}} = f_{\text{dir}} * F_{\text{in}} / \text{abs}(\mu_{\text{solar}})$
Symptom: Direct beam irradiance at canopy top is wrong by factor $ \mu_{\text{solar}} $.
Why: $\text{Irradiance} = \text{Intensity} * \mu $. To get intensity, divide by $ \mu $.

Error 2 — Ground BC missing the direct-beam term

Wrong: $F_{\text{down_gnd}} = F_{\text{d_dif}}[K+1]$
Correct: $F_{\text{down_gnd}} = F_{\text{d_dif}}[K+1] + F_{\text{d_dir}}[K+1]$
Symptom: Upward uncollided flux is too small. Reflectance is underestimated.
Slide 14 explicitly shows BOTH terms.
Recall: $F_{\text{d_dir}}[K+1] = \mu_{\text{solar}} * I_{0_dir}[K+1]$ (beam irradiance at ground).

Error 3 — Wrong sign convention for f in DD coefficients

Wrong: $f[n] = G * dL / \text{abs}(\mu[n])$ (positive for all n)
Correct: $f[n] = G * dL / \mu[n]$ (signed: negative for downward)
Symptom: Convergence is too fast (3-5 iters vs 8-15 for NIR).
Interior IC field is wrong. Scalar irradiance imbalance: -40% to -60%.
Why: With $f > 0$ for downward, the source J is SUBTRACTED not added.

Error 4 — Ground BC uses previous iteration's IC

Wrong: Compute $F_{\text{col_gnd}}$ from IC (previous iteration) before the sweep.
Correct: Compute $F_{\text{col_gnd}}$ from IC _{new} after the DOWNWARD sweep.
Symptom: Slower convergence. Slightly wrong interior field.

Why:	The ground BC should reflect the flux that arrives at the ground
	in the CURRENT sweep, not what arrived one iteration ago.

Error 5 — Single convergence criterion instead of dual

Wrong:	Stop when $\text{sum}(S_{\text{new}} - S_{\text{old}}) / \text{sum}(S_{\text{new}}) < \text{tol}$.
Correct:	Stop when BOTH crit_A (boundary IC) AND crit_B (SI profile) $< \text{tol}$.
Symptom:	NIR cases stop too early. Scalar irradiance imbalance: -40% to -60%.
Why:	The SI energy balance requires the interior field, not just the boundary.

Error 6 — Q2 or Q3 wrong direction range

Q2 sums $n=1..N/2$	(downward only), Q3 sums $n=N/2+1..N$ (upward only).
S sums $n=1..N$	(ALL directions).
	The OUTPUT direction $i=1..N$ for all of Q1, Q2, Q3, S (all hemispheres).

Error 7 — Wrong output direction in interpolation Q2

In interpolation (Steps 17-22), the output direction is $\Omega_V = (\mu_V, \phi_V)$,
NOT (μ_i, ϕ_j) from the Gauss grid.
All Gamma factors use $\Gamma(n \rightarrow \mu_V, \phi_V)$ as the output.

Error 8 — Double-counting Trans and Abs_soil in the energy balance

For a Lambertian ground: $F_{u_tot}[K+1] = \rho_g * F_{d_tot}[K+1]$
Therefore: $Trans = F_{d_tot}[K+1] - F_{u_tot}[K+1] = (1-\rho_g)*F_{d_tot}[K+1] = Abs_soil$
Trans and Abs_soil are THE SAME quantity.
Wrong: $Ref + Trans + Abs_leaves + Abs_soil = 1$ (gives ~1.33)
Correct: $Ref + Trans + Abs_leaves = 1$

Show Abs_soil in the table for physical completeness,
but do NOT include it in the SUM or imbalance formula.

15. Input Parameter Reference

Parameter	Typical value	Units	Physical meaning
N_MU	16	—	Polar Gauss-Legendre directions (must be even). N/2 downward + N/2 upward.
N_PHI	16	—	Uniform azimuthal directions.
N_LAY	50	—	Number of canopy layers K.
LAI_VALUES	[1.5, 4.0]	m ² /m ²	Leaf Area Index values to compute (list — all are run).
f_dir	0.70	—	Fraction of incoming flux that is direct solar (0=overcast, 1=full sun).
THETA_SUN_DEG	140.0	degrees	Solar polar angle from UPWARD zenith (must be >90 for downward sun).
PHI_SUN_DEG	0.0	degrees	Solar azimuth (0 = reference direction for principal plane).
rho_g	0.10 RED	—	Lambertian ground reflectance.
rho_L	0.06 RED	—	Leaf reflectance. Leaves absorb most RED, reflect most NIR.
tau_L	0.04 RED	—	Leaf transmittance. Small in RED, large in NIR (~0.45).
F_IN	1.0	W/m ²	Total incoming flux. Set 1.0 so outputs are automatically normalised.
TOL	0.01	—	Dual convergence tolerance. 0.01 = 1% relative change.
MAX_ITER	500	—	Safety limit on iterations. Should never be reached with TOL=0.01.
THETA_V_STEP	2	degrees	View polar step for BRF/HDRF interpolation grid.
PHI_V_STEP	4	degrees	View azimuth step for BRF/HDRF interpolation grid.

16. Code Structure Map

Section	Function name(s)	What it does
0	(input block)	All user inputs. START HERE to change parameters.
1	setup_quadrature(N,M)	Gauss-Legendre nodes/weights, azimuth grid, weight assertions.
2	G_extinction()	Returns G=0.5 for uniform leaf angle distribution.
3	gamma_scalar() precompute_gamma_quad() precompute_gamma_solar()	Phase function Gamma and pre-computed tables G_qq, G_sol.
4	solve_uncollided()	Steps 3a, 3b, 4, 5: all Beer-Lambert sweeps and ground BC.
5	compute_first_collision_source()	Step 6: Q = Q1+Q2+Q3 at all K cell centres, all N*M directions.
6	solve_collided()	Steps 7-10: DD coefficients, iteration loop with dual convergence.
7	compute_scalar_irradiance() compute_energy_balance()	Step 11: scalar irradiance S(k), then Ref, Trans, Abs, imbalance.
8	dom_interpolate_view()	Interpolation Steps 1-9 (C4-P4, Slides 18-23).
9	write_energy_balance_docx()	Writes energy balance table to Word document.
10	write_brf_hdrf_docx()	Writes BRF/HDRF angular table to Word document.
11	solve_one_case()	Orchestrates Sections 1-7 for one (band, LAI) pair.
12	main()	Entry point: loops over bands and LAI, asks for interpolation y/n.

Physical Models in Remote Sensing — Chapter 04, Parts 03 and 04

This document and dom_student.py are companion materials. Use them together.